

Linux Fundamentals

Note:

This is NOT a full Linux fundamentals course

 Terminal

Type commands

 Command

Instruction

Section A

Navigating the Filesystem

■ "Print working directory"

```
pwd
```

directory ⇔ folder

working directory ⇔ current directory

■ List files and directories

```
ls
```

```
ls {directory}
```

```
ls downloads
```

■ Note: `ls` does not change the working directory

 List all files and directories, including hidden ones

Using  `-a`

```
ls -a
```

 `-a` means "all"

```
ls -a downloads
```


■ Anatomy of a command line

{command}	{options...}	{arguments...}
-----------	--------------	----------------

--	--	--

ls	-a	downloads
----	----	-----------

■ "Change directory"

```
cd {directory}
```

```
cd downloads
```

Special directories

`..` points to the parent directory (one level up)

`.` points to the working directory

```
ls downloads/materials
```

A specific location of a file or directory is called a **path**

■ Relative path

No leading `/`

```
downloads/materials
```

```
../downloads
```

■ Absolute path

Starts with `/`

```
/home/username/downloads/materials
```

Practice (8 min)

Before you start, run `pwd` to see where you are currently at, and run `ls` to see the files inside where you are currently at.

1. Find the file named `bato.docx`.
2. Go to the directory where it is located.
3. See what files are in `wakanda/laboratory` without changing working directory.
4. Run `cd ~`. What directory are you in now?

Input	Meaning
<code>pwd</code>	"Print working directory"
<code>ls</code>	List files and directories
<code>cd</code>	"Change directory"
<code>..</code>	Parent directory

6. [Optional] While waiting for other participants, run `man ls` and explore (Press `q` to quit). Also try `man cd`.

man means manual.

 Show contents of a file

```
cat {file}
```

```
cat samples.txt
```

■ Page through a large file

```
less {file}
```

```
less output.txt
```

Use arrow keys to scroll. Press **q** to quit.

▨ Keep long lines from wrapping

Using **-S**

```
less -S output.txt
```


 Show first few lines

```
head {file}
```

```
head samples.txt
```

Default: First 10 lines

 Show first n lines

Using  `-n`

```
head -n {number_of_lines} {file}
```

```
head -n 3 samples.txt
```

```
{command} {options...} {arguments...}
```

|

|

|

```
head
```

```
-n=3
```

```
samples.txt
```

■ Show last few lines

```
tail sample_list.txt
```

```
tail -n 5 sample_list.txt
```

"Word count"

Shows number of lines, words, and bytes

```
wc {file}
```

```
wc pos.txt
```

Show number of lines only

Using  `-l`

```
wc -l pos.txt
```

■ Find lines matching a pattern

```
grep {pattern} {file}
```

```
grep chr1 pos.txt
```

Practice (4 min)

1. Go to Wakanda laboratory.
2. The file `output.txt` is long (don't ask me how I knew). Page through it with `less` (press `q` to quit).
3. Darna appeared out of nowhere and she wants to see only the first 12 output entries.
4. Each line has one entry. See the number of entries using `wc -l`.
5. If the count is not 100, the output must be incomplete. Use `tail` to quickly see the very last entry label recorded.

Input	Meaning
<code>cat</code>	Print the whole file
<code>less</code>	Page through a file
<code>less -S</code>	Do not wrap long lines
<code>head</code>	Show first few lines
<code>head -n {x}</code>	Show first x lines
<code>tail</code>	Show last few lines
<code>wc -l</code>	Count lines
<code>grep</code>	Show lines matching it

Section C

Managing Files

■ "Make directory"

```
mkdir {directory_name}
```

```
mkdir output
```


■ "Copy"

```
cp {source} {destination}
```

```
cp samples.txt output
```

```
cp downloads/ram.md .
```

```
cp samples.txt backup.txt
```

 Copy directories

Using 

```
cp downloads another_downloads
```

 means recursive

■ "Move"

```
mv {source} {destination}
```

```
mv backup.txt backup/snapshots
```

▨ Also used to rename a file or directory.

```
mv {file} {new_file_name}
```

```
mv ram.md extra-ram.md
```

 "Remove"

```
rm extra-ram.md
```

 Remove a directory (and files in it)

Using  `-r`

```
rm -r downloads-copy
```

■ Extract a `.tar.gz` archive

```
tar -xzvf {archive}
```

```
tar -xzvf demo.tar.gz
```

Flag	Meaning
-x	extract
-z	gunzip
-v	verbose
-f	file

Practice (8 min)

1. Extract the provided dataset `demo.tar.gz`, then go into the folder it creates.
2. Create an output directory named `results`.
3. Copy the file `results-summary.txt` into the `results` directory as `summary.txt`.
4. Rename the `results` directory to `results_run1`.
5. The run was cancelled. Delete `results_run1`.

Input	Meaning
<code>tar -xvzf</code>	Extract an archive
<code>mkdir</code>	"Make directory"
<code>cp</code>	"Copy"
<code>cp -r</code>	"Copy" a directory
<code>mv</code>	"Move" / rename
<code>rm</code>	"Remove"
<code>rm -r</code>	"Remove" a directory

Section D

Redirecting and Combining Commands

■ Output redirection

Send output to a *file* using `>`

```
[cmd] > {file}
```

```
ls > list.txt
```

⚠ `>` overwrites the file.

Pipes

Send output from one command into another using `|`

```
[cmd1] | [cmd2]
```

```
ls | head
```

Practice (4 min)

1. Count the lines in `pos.txt`, saving the result to `count.txt` with `>`.
2. Read `count.txt` to check what was saved.
3. Find lines in `pos.txt` that start with `1`, then show only the first 10 using a pipe.

Input	Meaning
<code>{cmd} > {file}</code>	Send output to a file
<code>{cmd1} {cmd2}</code>	Send output to input

Section E

Running Multiple Commands with Scripts

■ Print or echo back a text

```
echo {text...}
```

```
echo hi
```

```
echo hi everyone
```

■ Terminal

A program where you type commands.

■ Shell

A program that reads and runs commands.

We're using **bash**, which is a type of shell

■ Shell script

A file containing shell commands.

For example, this is a file named `run.sh`:

```
#!/usr/bin/env bash  
  
echo hi everyone!  
ls  
tail ~/samples.txt -n 1
```

▨ Run a script

```
./uploadDataset.sh
```



Make a script runnable

```
chmod +x {script}
```

```
chmod +x uploadDataset.sh
```

■ Comments

Add notes that are ignored by the shell using `#`

```
# This is a comment. This is not a command that runs anything.  
echo "This is a command"    # This prints something  
# echo "I will not run"
```


■ Passing arguments to a script

Suppose you run:

```
./hawak mo ang beat
```

Inside `hawak` file:

```
echo $0  
echo $1  
echo $2  
echo beauty and a $3
```

Output:

```
./hawak  
mo  
ang  
beauty and a beat
```

Variables

Stores a value that you can reuse

```
sample=pos.txt  
echo $sample preview  
head $sample
```

⚠ No spaces around =

```
sample=$1  
echo $sample preview  
head $sample
```

■ Variables from arguments

```
sample=$1      # from first argument
line_count=12  # hardcoded value

echo Checking first $line_count lines of $sample
head -n $line_count $sample
```

Practice (8 min)

1. Read the script named `check_data.sh`. Do not run it yet. Understand what it does.
 - Predict what it prints, line by line.
 - What must you give the script when you run it?
 - Which line will NOT print anything? Why?
2. Run it on `pos.txt`.

Input	Meaning
<code>#</code>	Comment (does not run)
<code>name=value</code>	Store value in variable
<code>\$name</code>	Use variable value
<code>\$1</code>	First argument passed
<code>\$2</code>	Second argument passed
<code>chmod +x</code>	Make a script runnable
<code>./{script}</code>	Run a script in folder

Common Mistakes

1. Running commands from the wrong folder
2. Accidentally overwriting files with `>`, `cp`, or `mv`
3. Using `rm` / `rm -r` carelessly
4. Adding spaces around `=` in shell variables

Why Linux?

1. Linux has strong tools for processing large data files.
2. Many bioinformatics workflows are designed for Linux.
3. Databases often live on servers.